Uppsala University

Parallel and Distributed Programming

1TD070

Assignment 3

Authors:
Anton von
Nandelstadh

May 10, 2026

1 Parallelisation

Algorithm 1 Parallel Global Sort

```
1: Divide the data into p equal parts
2: Sort the data locally in each processor
3: procedure GlobalSort(processors)
4:     Select pivot in each processor set
5:     for all processors do
6:         Divide the data into two sets (smaller or larger)
7:     end for
8:     Split the processors into two groups
9:     Exchange data pair-wise
10:    for all processors do
11:        Merge data into a sorted list
12:    end for
13:    Recursively apply GlobalSort to each processor group
14: end procedure
```

We implement the algorithm 1. The main loop simply reads the file, scatters the list to different nodes and calls global sort. Once global sort terminates, it gathers the information from each node, checks it, and prints it to the output file.

Listing 1: Core of main loop.

```
1     n = read_input(input, &elements);
2     local_n = distribute_from_root(elements, n, &
          my_elements);
3
4     double start = MPI_Wtime();
5     qsort(my_elements, local_n, sizeof(int), compare);
6     local_n = global_sort(&my_elements, local_n,
          MPI_COMM_WORLD, pivot_strategy);
7     double time = MPI_Wtime() - start;
8     double max_time;
9     MPI_Reduce(&time, &max_time, 1, MPI_DOUBLE, MPI_MAX,
          0, MPI_COMM_WORLD);
10
11    gather_on_root(elements, my_elements, local_n);
```

```
12      if (myid == 0) {
13          printf("%f\n", max_time);
14          check_and_print(elements, n, output);
15      }
```

Global sort starts by finding the pivot, and choosing its partner.

Listing 2: Exchanging data.

```
1       /* We select our pivot and get its index*/
2       int idx = select_pivot(pivot_strategy, *elements, n,
            communicator);
3
4       /*Split data into larger and smaller*/
5       int* v1 = *elements;
6       int* v2 = (*elements) + idx;
7
8       int n1 = idx;
9       int n2 = n - idx;
10
11      /*Exchange data pairwise between processors*/
12      int half = n_proc / 2;
13      int partner = (myid + half) % n_proc;
```

Once the pivot is known, we are able to exchange data between the node and its partner node.

Listing 3: Exchanging data.

```
1       // First we need to exchange list and pivot data
2       int data[3] = {n1, n2, idx};
3       int partner_data[3];
4       MPI_Sendrecv(data, 3, MPI_INT, partner, 0,
           partner_data, 3, MPI_INT, partner, 0,
           communicator, MPI_STATUS_IGNORE);
5
6       // Then we can actually exchange the lists
7       int* buf;
8       if (myid < half) {
9           int buf_n = partner_data[0];
10          buf = malloc(sizeof(int) * buf_n);
11          MPI_Sendrecv(v2, n2, MPI_INT, partner, 0, buf,
               buf_n, MPI_INT, partner, 0, communicator,
```

```
            MPI_STATUS_IGNORE);
        n2 = buf_n;
        v2 = buf;
    } else {
        int buf_n = partner_data[1];
        buf = malloc(sizeof(int) * buf_n);
        MPI_Sendrecv(v1, n1, MPI_INT, partner, 0, buf,
            buf_n, MPI_INT, partner, 0, communicator,
            MPI_STATUS_IGNORE);
        n1 = buf_n;
        v1 = buf;
    }
```

Once the node has its new data, it just needs to merge the lists and recurse.

Listing 4: Merging list and recursing.

```

    /*Merge the data into one list*/
    int* merged;
    merged = malloc((n1 + n2) * sizeof(int));
    merge_ascending(v1, n1, v2, n2, merged);
    *elements = merged;

    /*Recurse*/
    int color;

    if (myid < n_proc / 2) {
        color = 0;
    } else {
        color = 1;
    }

    MPI_Comm recursion_comm;
    MPI_Comm_split(communicator, color, myid, &
       recursion_comm);
    n = global_sort(elements, n1 + n2, recursion_comm,
       pivot_strategy);
    free(buf);
    MPI_Comm_free(&recursion_comm);
```

```
    return n;
```

We implemented the pivots as follows:

Listing 5: Pivots

```
int select_pivot_median_root(int* elements, int n,
   MPI_Comm communicator) {
    int myid, pivot;
    MPI_Comm_rank(communicator, &myid);
    pivot = get_median(elements, n);
    MPI_Bcast(&pivot, 1, MPI_INT, 0, communicator);
    return pivot;
}

int select_pivot_mean_median(int* elements, int n,
   MPI_Comm communicator) {
    int n_proc, pivot, sum;
    pivot = get_median(elements, n);
    MPI_Comm_size(communicator, &n_proc);
    MPI_Reduce(&pivot, &sum, 1, MPI_INT, MPI_SUM, 0,
       communicator);
    pivot = sum / n_proc;
    MPI_Bcast(&pivot, 1, MPI_INT, 0, communicator);
    return pivot;
}

int select_pivot_median_median(int* elements, int n,
   MPI_Comm communicator) {
    int n_proc, pivot, myid;
    pivot = get_median(elements, n);
    MPI_Comm_size(communicator, &n_proc);
    MPI_Comm_rank(communicator, &myid);
    int pivots[n_proc];
    MPI_Gather(&pivot, 1, MPI_INT, pivots, 1, MPI_INT,
       0, communicator);
    qsort(pivots, n_proc, sizeof(int), compare);
    pivot = get_median(pivots, n_proc);
    MPI_Bcast(&pivot, 1, MPI_INT, 0, communicator);
    return pivot;
}
```

2 Results

We ran our performance experiments on Pelle, using varying numbers of nodes. We wanted to measure both strong and weak scaling, so we did two sets of experiments.

However, first we needed to find which pivot to use. In order to do this, we simply compared the speedup on lists of length 125000000, both randomized and sorted in reverse order.

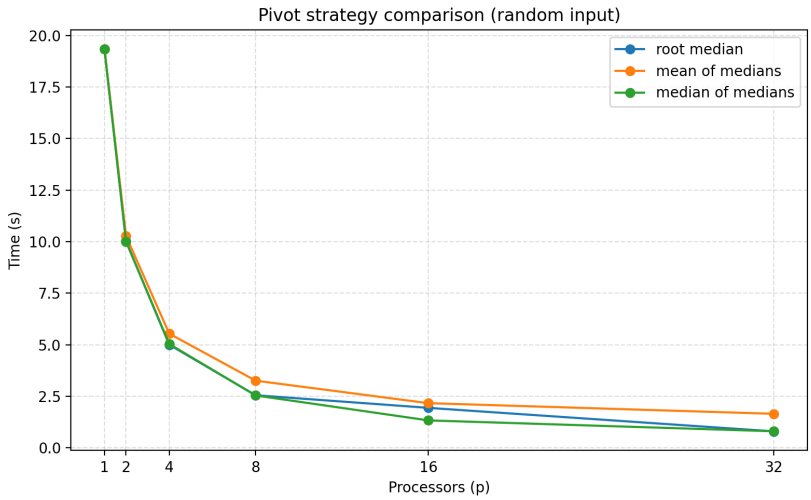

Figure 1: Pivot scaling on random data

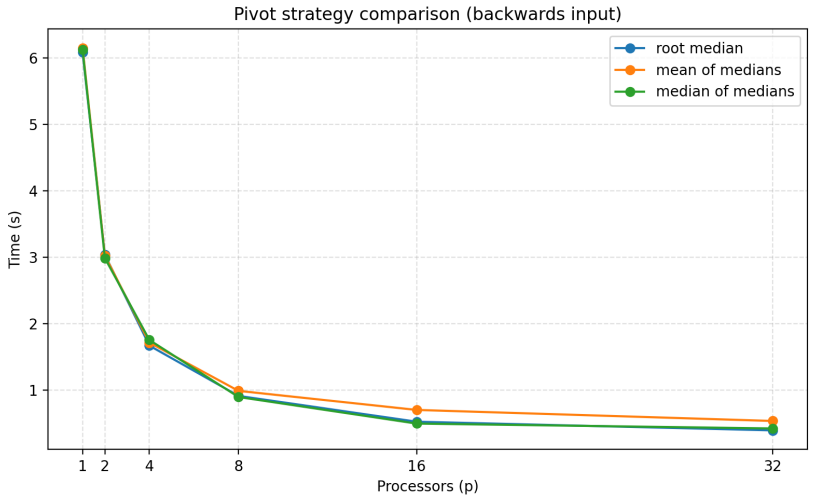

Figure 2: Pivot scaling on reversed data

We see that median of medians performs the very similarly to just taking the median on the root. They seem to outperform mean of medians on both randomized and reversed data. However, since the median of medians should theoretically be more robust, we choose to use it for our measurements of strong and weak scaling.

Once we found the pivot which we wanted to use, we computed strong and weak scaling. For strong scaling, we used a randomized list of length 2000000000. This can be seen in figure 3 and table 2. Second, we wanted to measure weak scaling, so we kept the input per core constant. That is, we increased input size as we increased the number of cores. This can be seen in figure 4 and table 1. In the table, p refers to the number of processes, n to problem size and h to the pivot. Here 1 is median on root, 2 is median of medians, and 3 is mean of medians.

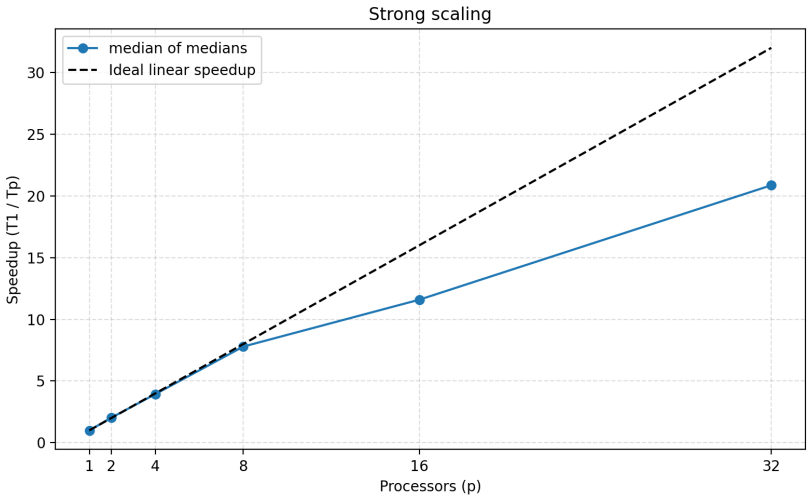

Figure 3: Strong scaling

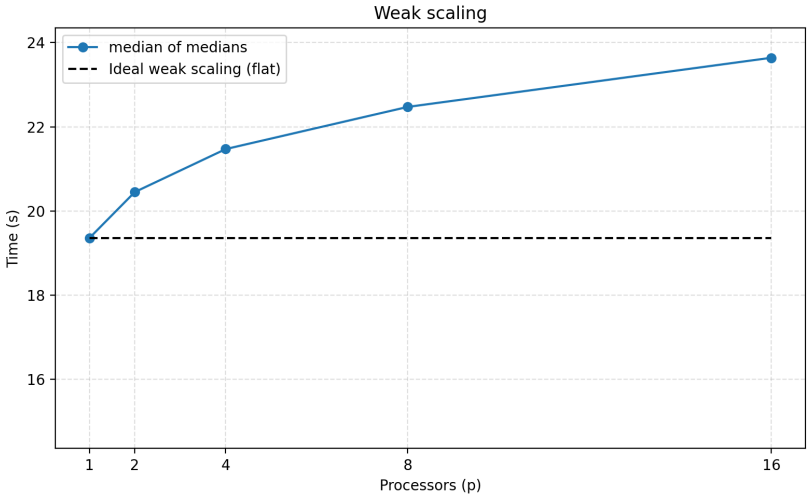

Figure 4: Weak scaling

n	p	h	time
125000000	1	3	19.354322
250000000	2	3	20.448796
500000000	4	3	21.470351
1000000000	8	3	22.471561
2000000000	16	3	23.640044

Table 1: Table for weak scaling.

n	p	h	time
2000000000	1	3	362.311640
2000000000	2	3	178.672842
2000000000	4	3	91.975455
2000000000	8	3	46.495825
2000000000	16	3	31.259098
2000000000	32	3	17.366399

Table 2: Table for strong scaling.

n	p	h	time
125000000	1	1	19.350250
125000000	2	1	10.017522
125000000	4	1	5.001278
125000000	8	1	2.550320
125000000	16	1	1.938625
125000000	32	1	0.797917
125000000	1	2	19.345551
125000000	2	2	10.271359
125000000	4	2	5.538521
125000000	8	2	3.257627
125000000	16	2	2.165803
125000000	32	2	1.650635
125000000	1	3	19.340827
125000000	2	3	10.007731
125000000	4	3	5.035831
125000000	8	3	2.539797
125000000	16	3	1.330049
125000000	32	3	0.802794

Table 3: Table for pivot on random data.

n	p	h	time
125000000	1	1	6.085865
125000000	2	1	3.039367
125000000	4	1	1.670545
125000000	8	1	0.914225
125000000	16	1	0.525301
125000000	32	1	0.394828
125000000	1	2	6.155022
125000000	2	2	3.027420
125000000	4	2	1.716102
125000000	8	2	0.989936
125000000	16	2	0.702779
125000000	32	2	0.536390
125000000	1	3	6.128511
125000000	2	3	2.989315
125000000	4	3	1.755920
125000000	8	3	0.897361
125000000	16	3	0.497175
125000000	32	3	0.423720

Table 4: Table for pivot on reversed data.

3 Discussion

The pivot selection did not seem to matter that significantly for the speedup achieved. However, on very pathological examples, the difference in speedup might be significant. We also saw quite some variation in timings between runs on Pelle, even with $sbatch --exclusive$ and $--bind-to\ cores$. However, this might have been due to ongoing security maintinance on Pelle at the time. For our final benchmarking, we chose to go with the median of medians as our pivot, since this should in theory be quite robust for different distributions of data.

Nevertheless, on very long lists of size 2×10^9, we achieved a speedup of around $\times 21$ on 32 cores. This is quite good. Limiting factors here are likely load imbalance between the processors, and the need to communicate with and wait on the partner nodes.